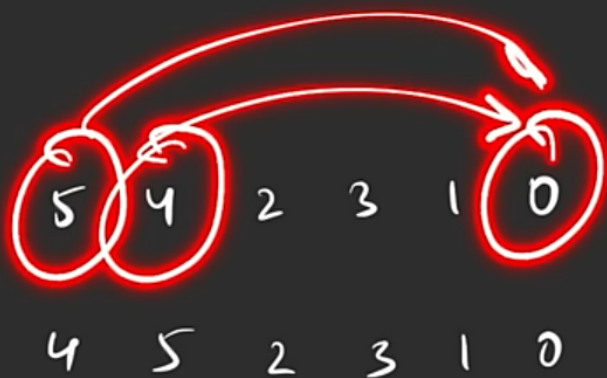
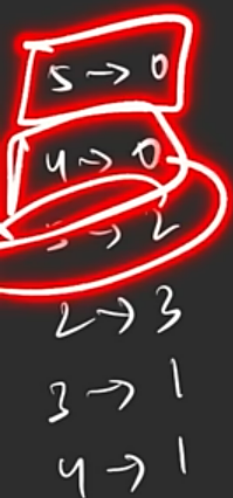
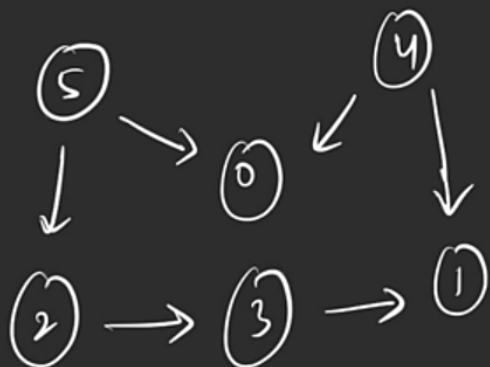
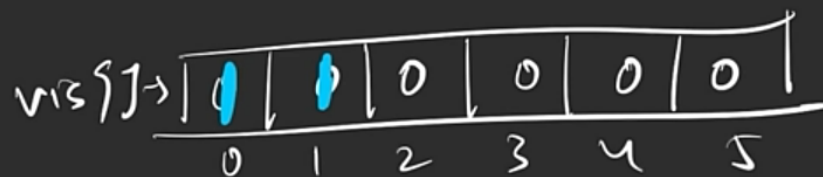
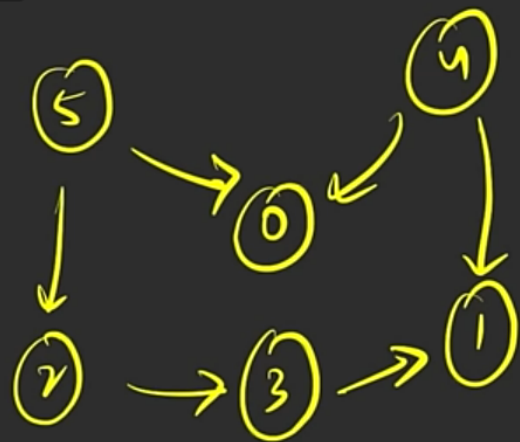
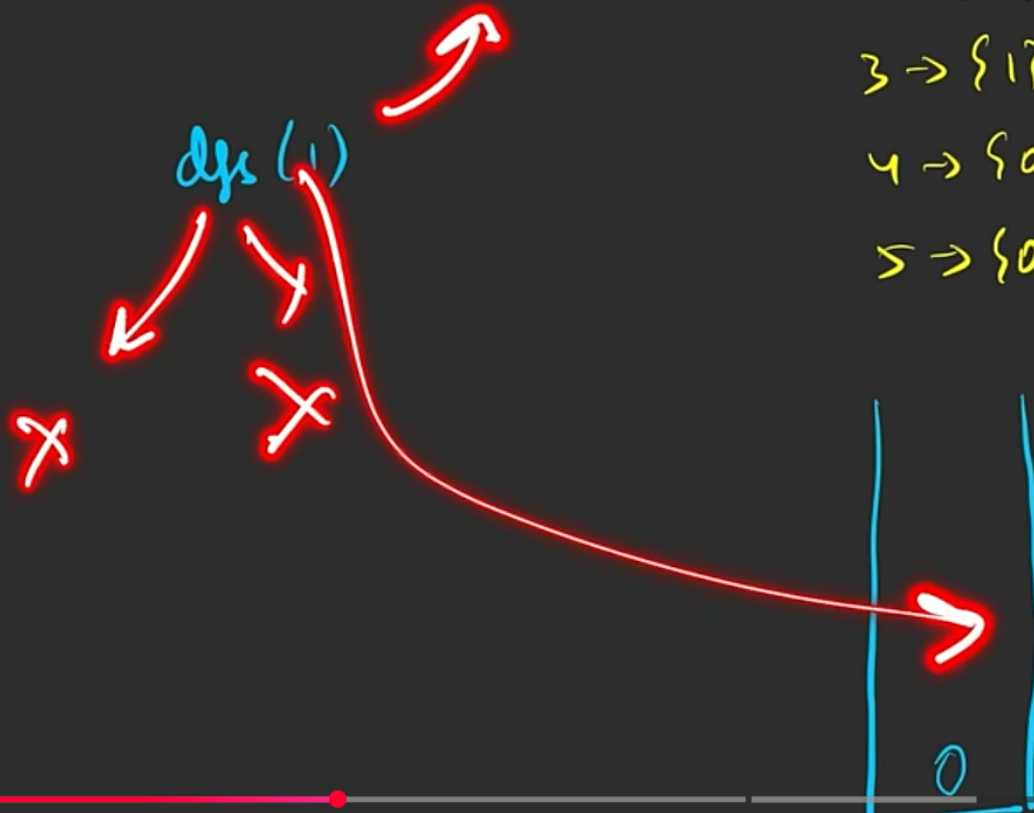
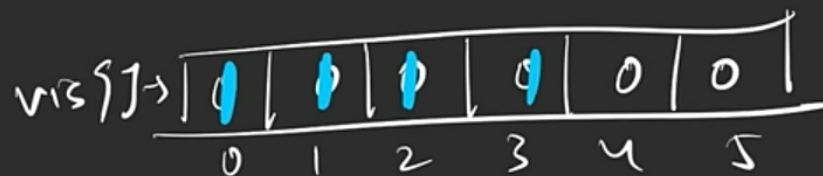
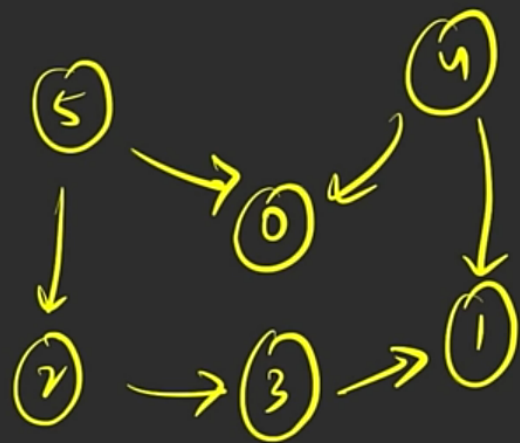


Topological Sorting (DFS)

↳ linear ordering of vertices such that if there is an edge between u & v , u appears before v in that ordering.



DFS $0 \rightarrow \{\}$ $1 \rightarrow \{\}$ $2 \rightarrow \{3\}$ $3 \rightarrow \{1\}$ $4 \rightarrow \{0, 1\}$ $5 \rightarrow \{0, 2\}$ $dfs(1)$ 

DFS $0 \rightarrow \{ \}$ $1 \rightarrow \{ \}$ $2 \rightarrow \{ \underline{3} \}$ $3 \rightarrow \{ \underline{1} \}$ $4 \rightarrow \{ 0, 1 \}$ $5 \rightarrow \{ 0, 2 \}$

dfs(2) ↗

↙ dfs(3) ↗

✗

2

3

1

0

(2) $\rightarrow$ (3) $\rightarrow$ (4)

{ 5 4 2 3 1 0 }

Topo Sort

one of the linear orderings

4 $\rightarrow$ {0, 1}

5 $\rightarrow$ {0, 2}

5 4 2 1 0

Stack

LIFO

1 2 3 4 5



```
for (i = 0 → 5)
{
    if (!vis[i])
    {
        dfs(i);
    }
}
```



</> Problem

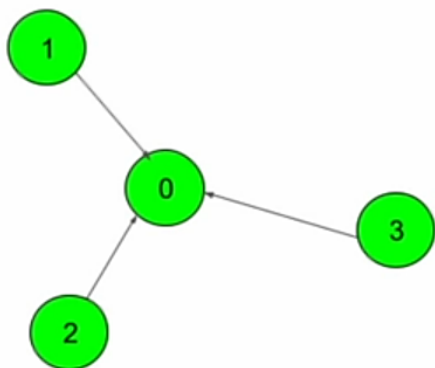
Editorial

Submissions

Comments

C++ (g++ 5.4)

Input:



Output:

1

Explanation:

The output 1 denotes that the order is valid. So, if you have implemented your function correctly, then output would be 1 for all test cases.

One possible Topological order for the graph is 3, 2, 1, 0.

Example 2:

```
1 // } Driver Code Ends
2
3 class Solution
4 {
5
6 private:
7     void dfs(int node, int vis[], stack<int> &st,
8         vector<int> adj[]) {
9         vis[node] = 1;
10        for(auto it : adj[node]) {
11            if(!vis[it]) dfs(it, vis, st, adj);
12        }
13        st.push(node);
14    }
15
16 public:
17     //Function to return list containing vertices in Topological order.
18     vector<int> topoSort(int V, vector<int> adj[])
19     {
20         int vis[V] = {0};
21         stack<int> st;
22         for(int i = 0; i < V; i++) {
23             if(!vis[i]) {
24                 dfs(i, vis, st, adj);
25             }
26         }
27         vector<int> ans;
28         while(!st.empty()) {
29             ans.push_back(st.top());
30             st.pop();
31         }
32         return ans;
33     }
34 };
35 // } Driver Code Ends
```